TECHNICAL WHITE PAPER

AI-Optimized PLL Test Time Reduction

Machine-learning methods for adaptive lock-time control, trim acceleration, jitter test elimination and sequential measurement in high-volume semiconductor production test

Scope	Integer-N / fractional-N PLLs, clock generators, SerDes and RF synthesizer PLLs tested on production ATE (wafer sort and final test)
Audience	Test engineering, product engineering, quality and data-science teams
Version	1.0 · 17 October 2026

All numerical results in this document come from synthetic data generated to illustrate the methods, or from clearly labelled example assumptions. They are not measurements from any specific product, tester or fab. Validate every method on your own data before use.

Contents

1. Executive summary . . . 3
2. PLL production test: where the time goes . . . 3
2.1 Typical PLL test content . . . 3
2.2 Why worst-case design wastes time . . . 4
3. Framework architecture . . . 4
3.1 Design principles . . . 5
3.2 Inference latency budget . . . 5
4. Technique A: adaptive lock-time control . . . 6
4.1 Principle . . . 6
4.2 Where AI helps . . . 6
4.3 Risks and controls . . . 6
5. Technique B: trim and calibration acceleration . . . 7
5.1 Predicted seed with narrow search . . . 7
5.2 Features for trim prediction . . . 7
5.3 Multi-parameter calibration . . . 7
6. Technique C: machine-learning test elimination . . . 7
6.1 Decision rule with conformal guard band . . . 7
6.2 Synthetic case study . . . 8
6.3 Practical guidance . . . 8
7. Technique D: sequential jitter and phase-noise measurement . . . 9
7.1 Stopping rule . . . 9
7.2 Where machine learning adds value . . . 9
8. Additional techniques . . . 10
8.1 Configuration and corner reduction (Technique E) . . . 10
8.2 Adaptive test ordering (Technique F) . . . 10
8.3 Alternate (signature) test (Technique G) . . . 10
9. Risk management and quality assurance . . . 10
9.1 Escape accounting . . . 10
9.2 Audit sampling and its limits . . . 10
9.3 Drift detection and automatic fallback . . . 11
9.4 Qualification and governance . . . 11
10. Data pipeline and implementation . . . 11
10.1 Data sources and feature engineering . . . 11
10.2 Reference decision logic . . . 12
10.3 Phased roadmap . . . 12
11. KPIs and return on investment . . . 13
11.1 Key performance indicators . . . 13
11.2 Worked example (illustrative assumptions) . . . 13
12. Conclusions . . . 13
Appendix A. Glossary . . . 14
Appendix B. Further reading . . . 14

1. Executive summary

Phase-locked loops are among the most time-expensive blocks to test on a mixed-signal SoC. Lock-time waits are sized for the slowest die at the worst corner, trim loops step through many codes, and jitter or phase-noise measurements need long captures to get statistically stable results. On many products the PLL portion of the flow accounts for a large share of total analog test time.

This paper describes an AI-optimized test framework that replaces fixed, worst-case test steps with **per-die decisions** driven by data the test flow already produces. Seven techniques are covered, ranging from low-risk changes that can be deployed in weeks (adaptive lock-time polling, predicted trim seeds, test reordering) to higher-value methods that require formal quality sign-off (machine-learning test elimination for jitter and phase noise, signature-based alternate test).

Key findings (synthetic study, Sections 4–6)
• Adaptive lock-detect polling cut mean lock-wait time by about 71% versus a fixed 60 µs worst-case wait, with no change to the pass/fail criterion.
• A gradient-boosted model predicted RMS jitter from cheap parametric tests with $R^2 = 0.86$. With conformal guard-banding and an outlier pre-screen, about 37% of parts could skip the jitter test within a 20 DPPM escape budget.
• Without the outlier pre-screen, escapes were dominated by defect-driven failures that parametric features cannot see. Defect screening, not model accuracy, is usually the limiting factor.
• In an illustrative budget, combining all techniques reduced PLL test time to about 41% of baseline.

The central engineering problem is not the model; it is **controlling escape risk**. The framework therefore pairs every skip or shortcut with a statistical guard band, a random audit sample, drift monitoring with automatic fallback to full test, and an explicit DPPM budget agreed with quality and customers. Section 9 covers this in detail.

2. PLL production test: where the time goes

2.1 Typical PLL test content

Test item	Typical method on ATE	Main time driver
Lock / lock time	Poll lock-detect bit or measure frequency settling after reset or frequency hop	Fixed worst-case wait per configuration
Frequency accuracy	Frequency counter or time-interval analyzer on divided output	Gate time for required resolution
VCO range and gain (K_{VCO})	Sweep band / control voltage, measure frequency	Number of bands and points
Trim / calibration	Binary or linear search over band, charge-pump, loop-filter or LDO codes	Iterations × settling per step
Period / cycle-to-cycle jitter	Time-interval capture, digitizer or on-chip jitter monitor	Samples needed for stable σ estimate
Phase noise / integrated jitter	Digitizer + FFT, or external analyzer	Capture length, averaging
Spurs and reference feed-through	Spectrum capture around carrier and offsets	Resolution bandwidth, averaging
Loop bandwidth / peaking	Reference modulation or step response	Multiple modulation frequencies
Configurations and corners	Repeat above across dividers, modes, VDD, temperature	Multiplicative test count

2.2 Why worst-case design wastes time

Conventional PLL test programs are written to be safe for every die. A lock wait that covers the slowest part at minimum VDD and cold temperature is applied to every part, even though the median die locks several times faster. Trim searches start from the middle of the code range even though wafer-level variation means most dies of a lot land in a narrow window. Jitter captures are sized so that a die sitting exactly at the limit is classified correctly, which is far longer than needed for a die with large margin.

Each of these is an opportunity: if the test program can **estimate where a die sits before or during the measurement**, it can spend test time only where the decision is uncertain.

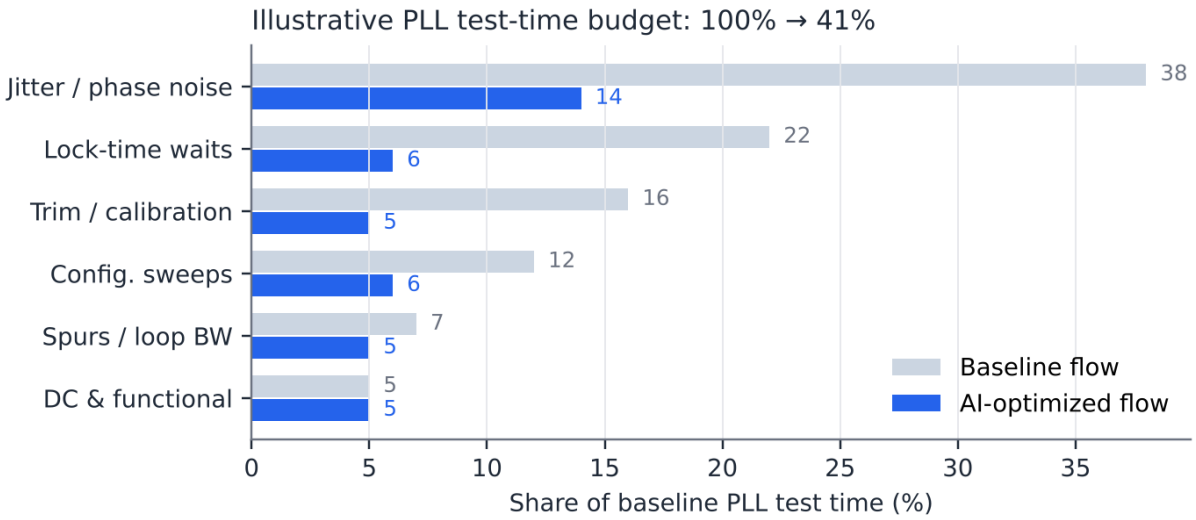


Figure 1. Illustrative PLL test-time budget before and after optimization. Shares are example values chosen to reflect a jitter-heavy flow; actual budgets vary widely by product.

3. Framework architecture

The framework has an offline side (data, features, training, validation) and an online side (inference on or near the tester, adaptive test program, monitoring). Figure 2 shows the components and data flows.

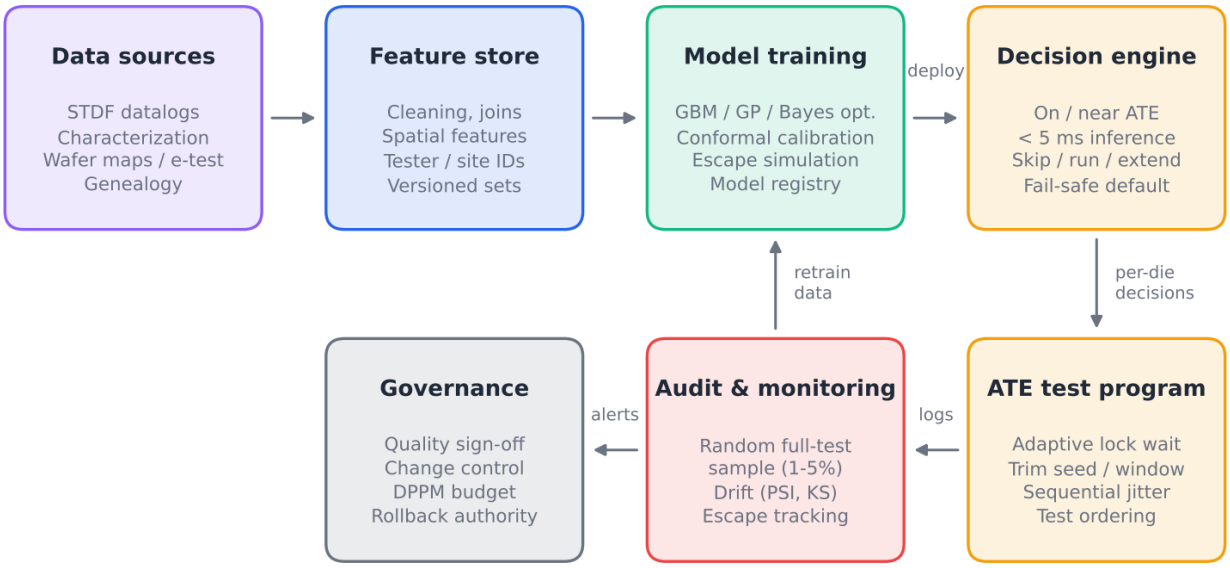


Figure 2. AI-optimized test architecture. The decision engine always has a fail-safe default: if inference is unavailable, late or out of distribution, the die receives the full baseline test.

3.1 Design principles

- **Fail-safe by default.** Any missing feature, model timeout, out-of-range input or drift alarm forces the full test. The optimized path is an exception that must be earned per die.
- **Decisions, not predictions.** The engine outputs one of *run*, *skip*, *shorten* or *extend*, together with the evidence (prediction, interval, model version) logged in the datalog for traceability.
- **Separation of concerns.** The ATE program exposes hooks (wait times, trim start codes, sample counts, test enables). Models never change limits.
- **Explainable models first.** Gradient-boosted trees and linear models with SHAP explanations are preferred over deep networks for tabular test data: they are accurate, fast, and easier to defend in a quality review.
- **Everything versioned.** Feature definitions, training sets, models and thresholds are versioned and tied to the test program revision under change control.

3.2 Inference latency budget

Per-die decisions must fit inside the test flow without adding visible overhead. Tree ensembles with a few hundred trees evaluate in well under a millisecond on a CPU. The dominant latency is usually data transfer between the tester workstation and the inference host. Practical options are: inference inside the test program (exported model as C code or ONNX runtime), on a tester-adjacent edge server, or through vendor real-time data platforms (for example Advantest ACS Real-Time Data Infrastructure, PDF Solutions Exensio, or Synopsys SLM). Feed-forward decisions that use data from earlier insertions (wafer sort → final test) have no real-time constraint at all.

4. Technique A: adaptive lock-time control

4.1 Principle

Replace the fixed wait T_{fixed} with polling of the lock-detect signal (or a frequency-settled check), followed by a short verification margin T_m. A timeout equal to the original wait preserves the original pass/fail behavior for slow dies.

$$T_{wait}(die) = \min(t_{lock}(die) + T_m , T_{fixed})$$

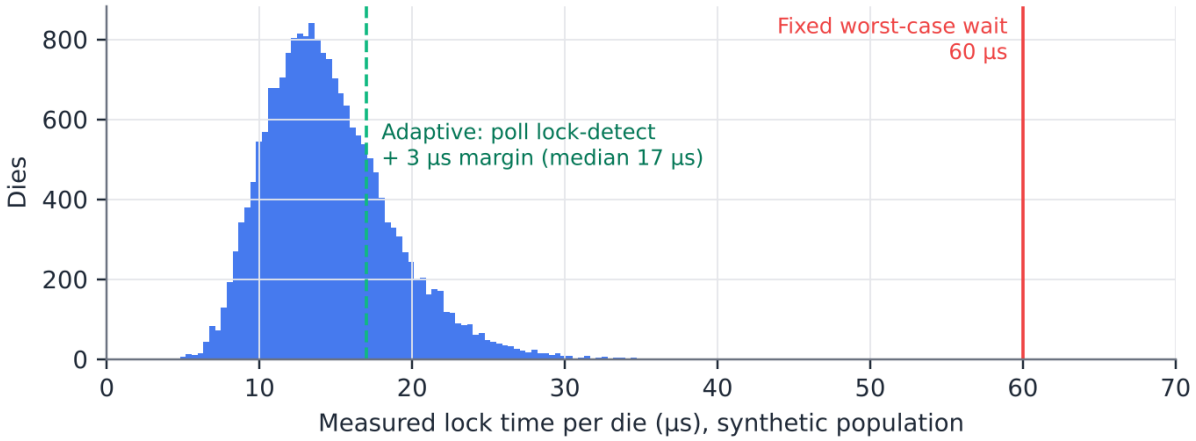


Figure 3. Synthetic lock-time distribution. Polling with a 3 µs margin reduces mean wait by about 71% compared with a 60 µs worst-case delay.

4.2 Where AI helps

- **Margin sizing.** Lock-detect asserts when phase error falls within a window, which does not guarantee frequency settling to the measurement accuracy. Train a model on characterization data (full settling waveforms) to predict the residual settling time after lock-detect as a function of configuration, VDD, temperature and parametrics. Use its upper prediction bound as T_m per configuration instead of a single global margin.
- **No lock-detect available.** Where the design has no usable lock indicator, predict t_{lock} from early transient samples (for example, the first few frequency-counter readings) using a regression model, and stop as soon as the predicted settled value is within tolerance with high confidence.
- **Outlier flagging.** Lock time itself is a strong health signal. Dies whose lock time is far from the predicted value for their parametrics should be routed to full test and flagged; abnormal lock dynamics often indicate defects in the charge pump or loop filter.

4.3 Risks and controls

The main risk is a die that asserts lock-detect early but has not yet settled in frequency or phase (for example, cycle slips after a fast initial acquisition). Control this by validating the margin on a full-waveform characterization set covering all corners, and by keeping a small audit sample that runs the original fixed wait.

5. Technique B: trim and calibration acceleration

5.1 Predicted seed with narrow search

A binary search over an n-bit trim code needs n measure-and-settle iterations regardless of where the answer lies. If a model can predict the final code $\hat{c}$ with uncertainty $\pm\delta$, the search only needs to cover the window $[\hat{c} - \delta, \hat{c} + \delta]$:

$$\text{iterations} \approx \lceil \log_2(2\delta + 1) \rceil + 1 \text{ (verify) versus } n$$

For a 6-bit VCO band code (64 bands) with a prediction accurate to ±2 codes, the search drops from 6 to about 4 iterations including verification, and often to 1–2 if the first predicted code is confirmed directly. If the verify step fails, the program falls back to the full search, so correctness is never compromised.

5.2 Features for trim prediction

- Wafer-level: x/y position, radial distance, reticle position, wafer and lot ID (as categorical or as lot-mean encodings from already-tested dies).
- Neighborhood: final trim codes of already-tested neighboring dies on the same wafer — often the single strongest predictor because of spatial process correlation.
- Process monitors: ring-oscillator frequency, leakage, bandgap / reference voltage, resistor and capacitor matching monitors.
- Upstream insertions: e-test (WAT) parameters, wafer-sort trim results when retrimming at final test.

5.3 Multi-parameter calibration

When several trims interact (for example, charge-pump current, loop-filter resistor and VCO band jointly setting loop bandwidth and jitter), sequential one-dimensional searches can need many iterations. Bayesian optimization with a Gaussian-process surrogate, warm-started with a prior learned from previous dies, can reach the target in fewer measurements. Because each measurement is expensive and the parameter space is small, this is a good fit for the method. Keep a hard iteration cap and a deterministic fallback.

6. Technique C: machine-learning test elimination

This technique offers the largest savings and carries the most risk. The goal is to predict the result of an expensive specification test (RMS or integrated jitter, phase noise at an offset, spur level) from inexpensive measurements, and skip the expensive test when the prediction is confidently inside the limit.

6.1 Decision rule with conformal guard band

Let $\hat{y}(\mathbf{x})$ be the model prediction for die features $\mathbf{x}$, and q_α the $(1 - \alpha)$ quantile of absolute residuals $|y - \hat{y}|$ on a held-out calibration set. Split conformal prediction guarantees, under exchangeability, that the true value lies within $\pm q_\alpha$ of the prediction with probability at least $1 - \alpha$. The skip rule for an upper limit USL is:

$$\text{skip} \Leftrightarrow \hat{y}(x) + q_\alpha < \text{USL} \wedge \text{die passes outlier pre-screen} \wedge \text{inputs in-distribution}$$

Two-sided limits use the analogous rule on both sides. Locally adaptive variants (normalizing residuals by a predicted spread, or conformalized quantile regression) give tighter intervals for dies in well-modelled regions and wider ones elsewhere, which usually increases the skip rate for the same escape budget.

6.2 Synthetic case study

A synthetic population of 200,000 dies was generated with a hidden process variable driving RMS jitter and six correlated cheap features (VCO band code, ring-oscillator frequency, charge-pump current, lock time, supply droop, radial wafer position). A small defect population (0.15%) adds large jitter that the process features cannot predict; a seventh feature emulates a cheap screen that partially detects defects (for example, loop-filter leakage or lock-detect glitch count). The overall jitter fallout was 0.44% at USL = 2.2 ps. A gradient-boosted regressor was trained on 30% of dies, calibrated on 20%, and evaluated on the remaining 50%.

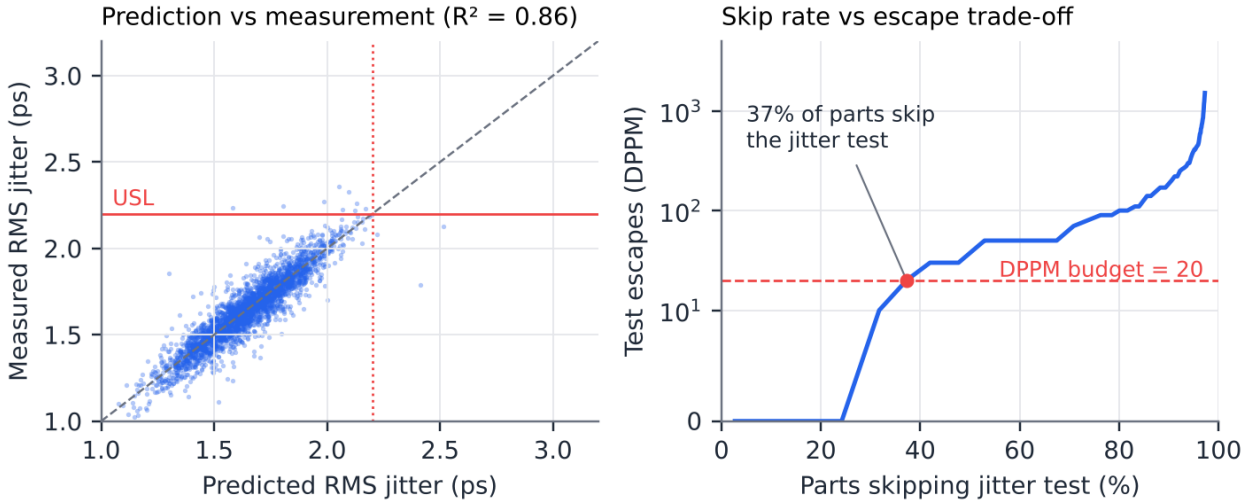


Figure 4. Left: predicted versus measured jitter (R^2 = 0.86). Right: share of parts skipping the jitter test versus resulting escapes as the conformal level α is swept. At a 20 DPPM budget about 37% of parts skip the test. Synthetic data.

Two lessons from this study carry over to real products:

- **Model accuracy is not the binding constraint.** Even with R^2 above 0.85, the first escapes came almost entirely from defect-driven parts. Without the outlier pre-screen, no setting of α met a 20 DPPM budget at a useful skip rate.
- **The skip-versus-escape curve is steep.** Moving from a 20 to a 100 DPPM budget roughly doubled the skip rate in this example. The DPPM budget is therefore a business decision with a large effect on savings, and should be set explicitly rather than inherited from a model default.

6.3 Practical guidance

- Start with the expensive test that has the **lowest fallout and highest correlation** to cheap tests. Tests that rarely fail but take a long time are ideal.
- Train on data where the expensive test was run on 100% of parts, spanning several lots, wafers, testers and probe cards. Hold out entire lots for validation, not random dies; random splits overstate performance because of lot-level correlation.
- Include tester, site and probe-card identifiers as features or as stratification variables, so site-to-site offsets do not masquerade as die variation.
- Use monotonic constraints in the boosted model where physics dictates the direction of an effect (for example, jitter not decreasing with lower charge-pump current). This improves robustness under drift.
- Consider classification (pass / fail / uncertain) instead of regression when the specification is a binary outcome or the measured value is heavily quantized.

7. Technique D: sequential jitter and phase-noise measurement

A jitter test estimates a standard deviation from *N* period or time-interval samples. *N* is normally fixed so that a die at the limit is classified with the required confidence. A die with large margin can be decided with far fewer samples.

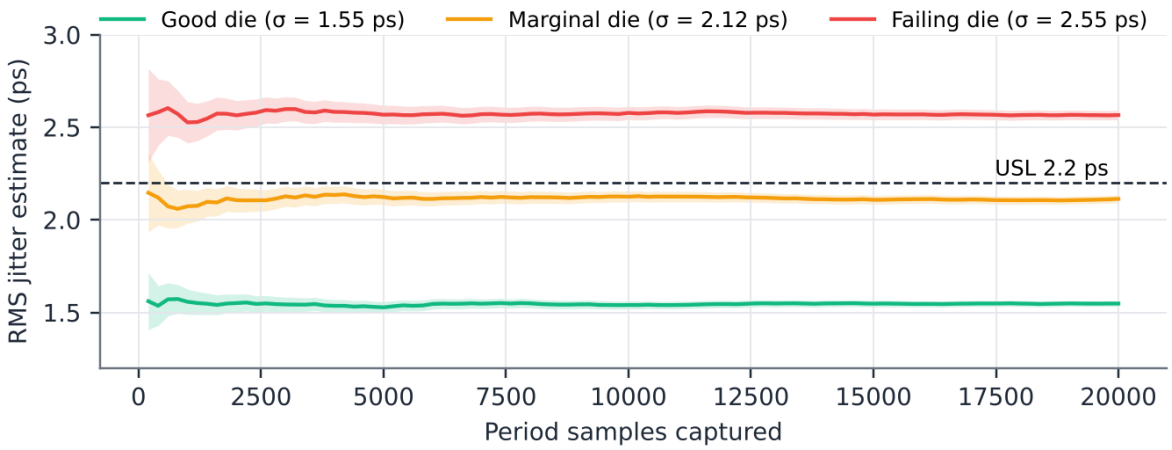


Figure 5. Running RMS-jitter estimate with 95% confidence band for three synthetic dies. Good and failing dies are clearly decided within a few hundred samples; only the marginal die needs the full capture.

7.1 Stopping rule

For Gaussian random jitter, the standard error of the sample standard deviation is approximately σ / √(2(*N* − 1)). Capture in blocks and stop when the confidence interval no longer contains the limit:

$$\text{stop at block k if } s_k\,(1 + z / \sqrt{2(N_k - 1)}) < \text{USL (pass) or } s_k\,(1 - z / \sqrt{2(N_k - 1)}) > \text{USL (fail)}$$

Because the rule is applied repeatedly, the per-look confidence level *z* must be inflated (for example with an alpha-spending function, or by using Wald's sequential probability ratio test, which is designed for repeated looks). Simulation on your own jitter distributions is the most reliable way to set *z*.

7.2 Where machine learning adds value

- **Prior-informed stopping.** Use the test-elimination model of Section 6 as a prior: a Bayesian update combining the predicted jitter with the first block of samples reaches a confident decision faster than the samples alone.
- **Non-Gaussian jitter.** Deterministic jitter, spurs and periodic components break the simple formula. Fit a dual-Dirac (RJ + DJ) or tail-fit model to the short capture and use the uncertainty of the extrapolated total jitter at the target BER as the stopping criterion.
- **Phase noise.** The same idea applies to FFT averaging: stop averaging when the integrated jitter or spot-noise estimate is confidently on one side of the limit.

8. Additional techniques

8.1 Configuration and corner reduction (Technique E)

When performance varies smoothly with divider ratio, output frequency or supply, a Gaussian-process or low-order surrogate fitted to a few measured configurations can predict the rest. The program measures a small anchor set, predicts the worst-case configuration, and measures that one directly. Configurations whose prediction interval approaches the limit are added adaptively. This keeps worst-case coverage while dropping configurations that carry no information for a given die.

8.2 Adaptive test ordering (Technique F)

Ordering tests by fallout per unit time minimizes expected test time for failing dies under stop-on-first-fail:

$$\text{priority}(t) = P(\text{fail}_t \mid \text{earlier passes, die features}) \,/\, T_t$$

The conditional fail probabilities can be estimated per lot, per wafer zone, or per die from upstream data, and updated as lots progress. Reordering must respect test dependencies (trim before specification tests, for instance) and should be limited to a small set of approved orders to keep the program qualifiable.

8.3 Alternate (signature) test (Technique G)

Apply a short, information-rich stimulus — a reference frequency step, a modulated reference, or a forced control-voltage perturbation — and capture the PLL response for a short time. A regression model maps the response signature to the full set of specifications (lock time, loop bandwidth, peaking, jitter). This approach has a long research history for analog and RF circuits. It offers large savings but requires a substantial characterization dataset and careful validation, so it is best introduced on a new product during characterization rather than retrofitted to a released flow.

9. Risk management and quality assurance

9.1 Escape accounting

For any skip or shortcut, the expected escape rate in DPPM is:

$$\text{DPPM}_{\text{esc}} = 10^6 \times P(\text{skip}) \times P(\text{fail} \mid \text{skip})$$

P(fail | skip) must be estimated on held-out lots during validation and tracked continuously in production through the audit sample. The total DPPM budget should be allocated across techniques (for example, 20 DPPM for jitter elimination, 5 DPPM for sequential measurement), and the sum agreed with quality and, where required, customers.

9.2 Audit sampling and its limits

A random fraction of skipped dies receives the full test. The audit sample size needed to observe at least one escape with confidence *C*, if the true escape rate among skipped parts is *p*, is:

$$n \geq \ln(1 - C) \,/\, \ln(1 - p)$$

Important limitation
Detecting an escape rate of 20 DPPM with 95% confidence needs about 149,785 audited parts. Per-lot audits therefore cannot verify DPPM-level budgets directly. Audits should be pooled across lots, and complemented by **margin monitoring**: track the distribution of measured-minus-predicted residuals on audited parts, which reveals model degradation long before escapes become observable.

9.3 Drift detection and automatic fallback

- **Input drift.** Population stability index (PSI) or Kolmogorov–Smirnov tests on each feature per lot and per wafer; any feature outside its training range for a given die forces full test for that die.
- **Residual drift.** Control charts (EWMA or CUSUM) on audit residuals; a shift beyond limits suspends skipping for the product until reviewed.
- **Context changes.** New tester, probe card, load board, test program revision, mask revision or fab/assembly site automatically returns the flow to full test until the model is revalidated on new data.
- **Yield excursions.** Lot yield below a threshold, or abnormal fallout on correlated cheap tests, disables optimization for that lot.

9.4 Qualification and governance

- Run the decision engine in **shadow mode** (predict and log, but run every test) for a defined number of lots before enabling any skip.
- Document model scope, training data, validation results, guard-band settings and fallback logic in a model card stored with the test program.
- Treat model updates as test program changes: version, review, and release through the same change-control process.
- For automotive (AEC-Q100, IATF 16949) and functional-safety products, involve the customer early. Many customers require notification or approval for test reductions, and some prohibit eliminating specification tests outright; adaptive lock time, trim seeding and sequential measurement are usually easier to qualify because the measurement criterion itself is unchanged.

10. Data pipeline and implementation

10.1 Data sources and feature engineering

Source	Examples	Use
Production datalogs (STDF)	Parametric results, trim codes, lock times, bin, site, tester, timestamps	Primary features and labels
Characterization	Full settling waveforms, multi-corner sweeps, long jitter captures	Training labels for margins, surrogate models
Wafer-level data	Die x/y, reticle, e-test / WAT, inline metrology	Spatial and process features
Upstream insertions	Wafer-sort results for final test	Feed-forward features
Equipment context	Tester, probe card, load board, handler, temperature log	Stratification, drift checks

Spatial features are frequently the strongest predictors: neighbor-die averages of the target and of key parametrics, radial distance, and wafer-zone encodings. Compute only features available at decision time; leakage from later tests is the most common cause of models that look excellent offline and fail in production.

10.2 Reference decision logic

The listing below shows the per-die decision logic for jitter test elimination. It is deliberately simple: every branch that is uncertain returns the full test.

```
import numpy as np

class JitterSkipEngine:
    """Per-die decision: 'SKIP' or 'RUN' for the RMS jitter test."""

    def __init__(self, model, q_alpha, usl, feat_ranges, leak_gate, version):
        self.model = model            # trained regressor (e.g. LightGBM / XGBoost)
        self.q = q_alpha              # conformal residual quantile at chosen alpha
        self.usl = usl                # upper spec limit (ps)
        self.ranges = feat_ranges     # {feature: (min, max)} from training data
        self.leak_gate = leak_gate    # outlier pre-screen threshold
        self.version = version

    def decide(self, feats: dict, enabled: bool = True):
        log = {"model": self.version}
        if not enabled:                                  # lot / drift kill switch
            return "RUN", {**log, "reason": "disabled"}
        if any(feats.get(k) is None for k in self.ranges):
            return "RUN", {**log, "reason": "missing_feature"}
        for k, (lo, hi) in self.ranges.items():          # out-of-distribution guard
            if not lo <= feats[k] <= hi:
                return "RUN", {**log, "reason": f"ood:{k}"}
        if feats["leak_screen"] >= self.leak_gate:       # defect pre-screen
            return "RUN", {**log, "reason": "outlier"}
        x = np.array([[feats[k] for k in self.ranges]])
        y_hat = float(self.model.predict(x)[0])
        upper = y_hat + self.q
        log.update(pred=round(y_hat, 4), upper=round(upper, 4))
        if upper < self.usl:
            return "SKIP", {**log, "reason": "confident_pass"}
        return "RUN", {**log, "reason": "uncertain"}

# Audit: a seeded random draw forces RUN on a fixed share of SKIP decisions,
# and the full measurement is logged next to the prediction for residual tracking.
```

10.3 Phased roadmap

Phase	Scope	Duration (typical)	Exit criteria
0. Baseline	Test-time profiling per test and configuration; data pipeline from STDF to feature store	2–4 weeks	Time budget known; clean, joined datasets
1. Low-risk wins	Adaptive lock polling, trim seeding with verify, test reordering	4–8 weeks	Savings measured; zero change in yield or bin distribution on audit lots
2. Shadow mode	Test-elimination and sequential models run in parallel with full test	8–12 weeks / 20+ lots	Escape estimate within budget on held-out lots; drift monitors validated
3. Controlled release	Enable skipping on one product and tester fleet with elevated audit rate	1–2 quarters	Quality and customer sign-off; audit residuals stable
4. Scale	Extend to more products; automate retraining and monitoring	Ongoing	Standard operating procedure in place

11. KPIs and return on investment

11.1 Key performance indicators

- PLL test time per die and total test time per die (mean and by bin).
- Throughput (units per hour) and tester utilization.
- Skip rate and shortened-test rate per technique.
- Estimated and audited escape DPPM; field returns attributable to skipped tests.
- Audit residual statistics (mean, spread, control-chart status).
- Fallback rate (share of dies routed to full test, and why).

11.2 Worked example (illustrative assumptions)

Parameter	Value	Note
PLL test time per die (baseline)	1.2 s	Assumption
Total test time per die (baseline)	4.0 s	Assumption
PLL time reduction	59%	From Figure 1 budget (100% → 41%)
Time saved per die	0.71 s	18% of total test time
Annual volume	50 million units	Assumption
Effective test cost	$0.03 per die-second	Assumption, includes multi-site and depreciation
Gross annual saving	≈ $1.06 million	Before implementation and audit costs

Implementation costs include data engineering, model development, test program changes, the audit sample (which consumes part of the saved time), and ongoing monitoring. Lower test time can also defer tester capital purchases, which is often a larger benefit than the per-unit cost saving.

12. Conclusions

PLL test time is dominated by steps designed for the worst-case die. Machine learning allows the test program to estimate, per die, how much measurement is actually needed to reach a confident decision. The recommended path is to start with techniques that keep the measurement criterion unchanged — adaptive lock polling, predicted trim seeds, sequential stopping and test ordering — and then introduce test elimination for jitter and phase noise once the data pipeline, monitoring and governance are proven.

Success depends less on model sophistication than on disciplined risk control: conformal guard bands, outlier pre-screens, lot-held-out validation, pooled audits, drift monitoring with automatic fallback, and an explicit DPPM budget agreed with quality.

Appendix A. Glossary

Term	Meaning
ATE	Automatic test equipment
DPPM	Defective parts per million shipped
STDF	Standard Test Data Format, the common ATE datalog format
Conformal prediction	Distribution-free method that turns any model into prediction intervals with guaranteed coverage under exchangeability
GBM	Gradient-boosted machine (e.g. XGBoost, LightGBM)
GP	Gaussian process, a probabilistic surrogate model
PSI	Population stability index, a drift metric
SPRT	Sequential probability ratio test (Wald)
RJ / DJ	Random jitter / deterministic jitter
SHAP	Shapley additive explanations, a feature-attribution method
Shadow mode	Model runs and logs decisions without affecting the test flow

Appendix B. Further reading

- P. N. Variyam, S. Cherubal, A. Chatterjee, "Prediction of analog performance parameters using fast transient testing," *IEEE Trans. CAD*, 2002.
- S. Biswas, R. D. Blanton, "Statistical test compaction using binary decision trees," *IEEE Design & Test of Computers*, 2006.
- H.-G. Stratigopoulos, Y. Makris, "Error moderation in low-cost machine-learning-based analog/RF testing," *IEEE Trans. CAD*, 2008.
- A. Wald, "Sequential tests of statistical hypotheses," *Annals of Mathematical Statistics*, 1945.
- V. Vovk, A. Gammerman, G. Shafer, *Algorithmic Learning in a Random World*, Springer (conformal prediction).
- Y. Romano, E. Patterson, E. Candès, "Conformalized quantile regression," *NeurIPS*, 2019.
- AEC-Q100 and AEC-Q001 (part average testing) for automotive test-flow requirements.
- M. P. Li, *Jitter, Noise, and Signal Integrity at High-Speed*, Prentice Hall (dual-Dirac jitter modelling).